

Acoular Workshop: Accessible and Reproducible Microphone Array Signal Processing with Python

Ennes Sarradj¹, Mikolaj Czuchaj¹, Gert Herold¹, Adam Kujawski¹, Oliver Lylloff², Art Pelling¹

¹ *Fachgebiet Technische Akustik, TU Berlin, 10587 Berlin, Germany, Email: ennes.sarradj@tu-berlin.de*

² *DTU WIND Department of Wind and Energy Systems, 4000 Roskilde, Denmark*

Introduction

Acoular is an open-source Python package designed for analyzing multichannel signals acquired from microphone arrays and other sensor arrays. It is a powerful tool for researchers and engineers involved in acoustic signal processing, has attracted a global user base and has continuously evolved with new features and improvements. Acoular is widely used in various fields, including aeroacoustics, automotive noise analysis, environmental noise monitoring, and bioacoustics. The software provides a comprehensive set of tools for beamforming, deconvolution, source localization, and signal analysis, making it a versatile platform for acoustic research and industrial applications. Acoular is built on a modular and extensible architecture, allowing users to customize and extend its functionalities to suit their specific needs. The software is actively maintained and supported by a dedicated team of developers and researchers, ensuring its continued relevance and usability in the field of acoustics. This article provides an overview of Acoular's key features, design and applications. It also serves as an introduction to the seven other contributions at the Acoular workshop at DAS DAGA 25 [1].

Short history of Acoular

Acoular is a relatively mature software package that has undergone significant evolution since its inception. Some selected milestones in the development of Acoular are highlighted below to illustrate its growth and transformation over the years. Development of Acoular began in 2006 as a library for aeroacoustic wind tunnel testing using microphone arrays with beamforming and advanced frequency-domain algorithms. It could be used in batch processing measured data as well as for measurement control and interactive analysis using a Qt-based graphical user interface. In the following years, Acoular has transitioned from a specialized tool for wind tunnel testing to a versatile platform capable of addressing a wide range of acoustic array signal processing challenges. In 2009, time-domain algorithms were introduced for the characterization of freely moving sources, and applied for the measurements aeroacoustic sound generation in flying birds (Fig.1). The same year saw the incorporation of multichannel signal synthesis capabilities, enabling the generation of synthetic acoustic data for testing and validation purposes.

By 2010, Acoular had implemented 3D mapping capabilities for wind tunnel applications analyzing the sound generation (for example on a high-speed train pantograph, Fig.2). The year 2014 saw the development of virtual

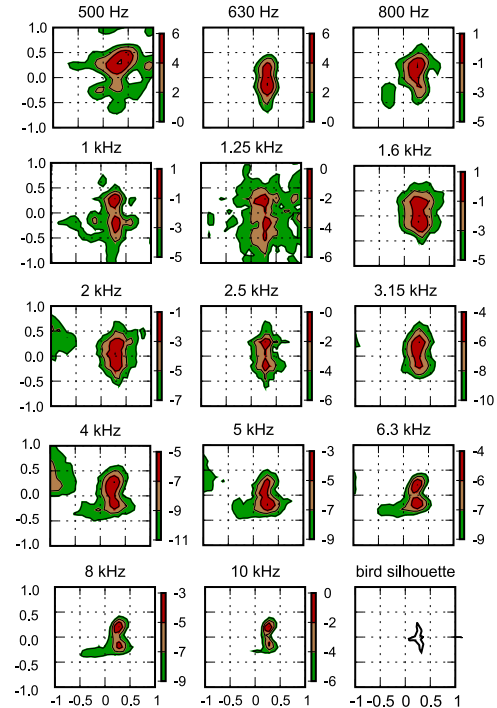


Figure 1: Third octave band sound pressure level map of a flyover of a Common Kestrel (*Falco tinnunculus*) measured in 2009 with a 92 microphone array (adapted from [2]).

rotation methods for axial fan analysis. Acoular was officially released as an open-source package in 2015, and has been consistently expanded and updated since then. In 2019, the SpectAcoular package was launched, providing a browser-based GUI (Fig.3 shows an example). Two years later, in 2021, the Acoupipe package was introduced to support deep-learning data provision. Most recently, in 2024, Acoular secured a grant from the German Research Foundation (DFG), further supporting its ongoing development.

Applications of Acoular

Acoular's flexibility enables its application in various scenarios across academia and industry. It is used for tasks such as noise source identification in aerospace applications, automotive noise studies, as well as in fields like bio-acoustics and underwater acoustics. The software provides tools and methodologies for detailed and accurate signal analysis. This is best illustrated by the following examples of Acoular's features in different fields.

Acoular can be used for source localization and characterization in the frequency domain. It includes multiple

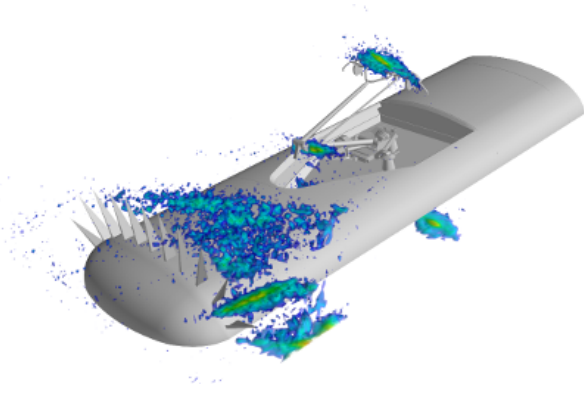


Figure 2: 4 kHz octave band sound map of a pantograph with turbulator spires from 2010, dynamic range 30 dB, wind speed 280 km/h (from [3]).

beamforming methods, which allow for identification of sound sources. The software also provides deconvolution techniques to enhance the resolution of acoustic maps and enable quantitative results from measurements. Additionally, a number of inverse methods are available, offering diverse tools for comprehensive acoustic analysis.

Another strong suit of Acoular is its support for the analysis of moving sources. It offers multiple beamforming and deconvolution methods for time-domain analysis, which are particularly useful for flyover and pass-by measurements. The software can also account for refraction in arbitrary flow environments, making it highly applicable to wind tunnel applications. Acoular also supports rotating sources, such as fans and other turbomachinery, and can analyze them effectively using either virtual array rotation or other approaches.

The software can also generate synthetic multichannel acoustic data, enabling the simulation of various types of acoustic sources, including moving sources. This is particularly useful for testing and validating signal processing algorithms, as well as for training machine learning models. Acoular’s synthetic data generation capabilities are complemented by its support for training data-driven models through Acoupipe.

Acoular extends its utility beyond traditional applications to areas such as environmental noise analysis, animal vocalization studies, and seismic wave analysis. Its versatility makes it applicable across various disciplines. The software also supports advanced multichannel signal processing tasks, including cross power spectra analysis, filter banks, frequency weighting, time weighting, and averaging. The SpectAcoular sound level meter app exemplifies these capabilities. Additionally, Acoular enables measurement hardware control, allowing data streaming from devices and providing a graphical user interface for visualization and analysis.

Design of Acoular

From the start, Acoular was designed to be easily adaptable and extendable. It uses an object-oriented programming paradigm that encapsulates implementation details of processing steps and offers the possibility to easily add

new processing capabilities through the definition of new classes that inherit most of the necessary code from already existing classes. For instance, the implementation of a new beamforming method can be done by defining a new class that inherits from an existing class and only needs to redefine those processing steps that are different from the existing method. This design allows for a high degree of code reuse and minimizes the effort required to add new functionality to the software. Overall, Acoular has approximately 100 different classes covering different functionalities such as data acquisition and data import, time-domain linear and nonlinear filtering, spectrum estimation, array layout and mapping grids definition, spatial filtering (beamforming) and deconvolution with different methods in both time and frequency domain, inverse estimation methods, the ability to account for different sound propagating environments, fixed and moving sources and data synthesis.

The object-oriented approach also allows for a lean user interface that is easy to navigate and understand. In script-based use, a signal processing problem can be stated with a few lines of code. This makes Acoular accessible to users with varying levels of programming experience and utilizes the easy access the Python as a programming language offers. However, building on the Python programming language also comes with some possible drawbacks. Python is an interpreted language and can be slower than compiled languages like C or Fortran, while multichannel signal processing and array methods in particular require a lot of computational resources. Moreover, in some cases huge amounts of data have to be processed, which can be a challenge. Acoular therefore includes some facilities for fast processing of large data sets, such as the use of the Numba library for just-in-time compilation of Python code to machine code optimized for speed. Another factor contributing to speedy processing is the use of multiple processing cores where the hardware supports this. Three more indirect factors enable efficient use of computing resources and therefore speed up processing: Acoular uses the lazy computing paradigm throughout, which means that processing steps are only executed when the results are actually needed (Only compute the result that is actually needed). This is particularly useful when processing steps are chained together, as it avoids unnecessary intermediate results. The second factor is the use of memory-mapped files for data storage, which allows for efficient access to large data sets without the need to load them into memory completely. Finally, Acoular has intelligent caching mechanisms that store final and intermediate results to avoid redundant computations. These mechanisms are particularly useful when processing steps are repeated with the same data set (Never compute the same thing twice!).

While using just-in-time compilation, Acoular is a pure python package and can be run on many different platforms that support Python. This includes Windows, Linux, and MacOS operating systems. The software is compatible with a wide range of hardware setups, from personal laptops to high-performance computing clusters

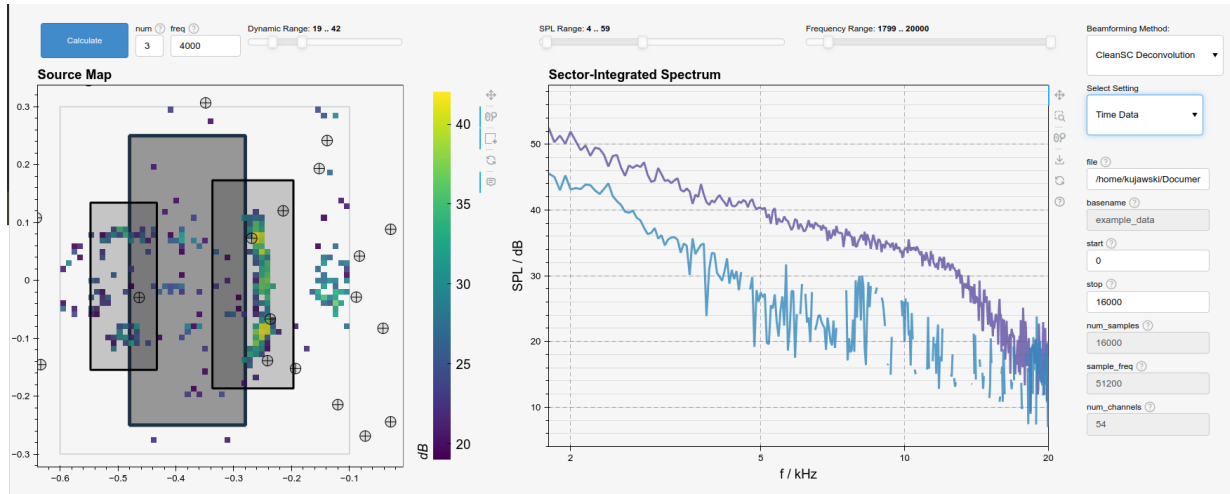


Figure 3: SpectAcoular based app to analyze sound maps and spectra

and even edge devices, such as the Raspberry Pi. Acoular supports multiple versions of Python, currently ranging from 3.10 to 3.13.

Access to Acoular

Acoular itself is available from Github, PyPi and other repositories. The software is distributed under a permissible new BSD style license, which allows for free use, modification, and distribution of the software. This open-source approach encourages use and contributions from the community. The software is accompanied by extensive documentation, which includes numerous examples and tutorials to help users get started. A large part of the documentation is automatically built from the source code comments, which ensures that it is always up-to-date. Support is also available through an active discussion forum, where users can ask questions and share their experiences with the software. Acoular is under active development on GitHub and is open to contributions from the community. To support contributions, a developer guide and a code of conduct are available on the project's website.

When Acoular was made available openly, it received immediate attention in the research community concerned with array based acoustic testing. Meanwhile, Acoular is successfully applied in many different research groups for different purposes. While it is difficult to give a complete record of its use, there are some general indicators that reveal the awareness of the community. The paper on Acoular [4] from 2017 is frequently cited, with most of the citations coming from others than the primary developers. The Acoular Github repository¹ received significant attention and, despite being niche-software, was starred over 500 times at the time of writing.

Some examples of the use of Acoular in research by others than the primary developers are:

- processing of microphone array data from aeroacoustic wind tunnel measurements: research group at the University of Bristol, UK [5, 6, 7], research

group composed from the Universities of Cambridge, Southampton, Cottbus and Brunel University, [8], and research group at the University of New South Wales, Australia [9, 10]

- processing of propeller noise data at Von Karman Institute, Belgium [11]
- hidden vehicle detection at TU Delft, The Netherlands [12]
- synthetic data generation for development of deep learning array methods at Chongqing University, China [13] and at Xiamen University, China [14]
- benchmarking for the development of algorithms to deal with rotating sources at Shanghai Jiao Tong University, China [15, 16, 17]

Acoular also finds its use in teaching and related activity. Some reports² of its use to realize DIY ‘acoustic cameras’ are available. At the University of Santa Maria, Brazil, Acoular is part [18] of the curriculum in Engineering Acoustics.

Quality assurance and reproducibility

As a software for scientific use, Acoular places a strong emphasis on quality assurance and reproducibility. Benchmarking with other software packages is challenging, as many of them are not open-source and therefore not readily available for comparison. However, Acoular has successfully participated in two benchmarking exercises [19, 20], one of which included another open source software package, *aeroacoustics.jl* [21], which is implemented in the Julia programming language.

The software is tested using continuous integration pipelines that run automated tests on different platforms and Python versions. Unit tests are used to verify the correctness of individual components. Integration tests ensure that the results produced from Acoular stay the same over different versions of Acoular itself, the operating

¹<https://github.com/acoular/acoular>

²e.g. at <https://navat.substack.com/p/diy-acoustic-camera-using-uma-16>

system and Python packages Acoular depends on. Additionally, linting is used to ensure that the code adheres to the style guide. This ensures that the code is readable, maintainable, and free from common programming errors.

Acoular uses git-based version control to track changes and releases. Changes are introduced only through pull requests, which are reviewed by other developers before being merged. This ensures that the code is of high quality and that changes are well-documented. The software follows a versioning with regular releases, which ensures that users can easily understand the impact of a new release on their workflows. Acoular also maintains a changelog that summarizes the changes in each release, making it easy for users to stay up-to-date with the latest developments.

Conclusion

This contribution contains an overview about Acoular, an open Python package for array signal processing. Acoular is a mature library, designed for performance, ease of use and extensibility. Both accessibility and reproducibility are important aspects in the design and application of Acoular.

References

- [1] E. Sarradj, G. Herold, A. Kujawski, A. J. R. Pelling, M. Czuchaj, J. Tschavoll, O. Lylloff, and R. Eschenhagen. *Acoular Workshop – DAS – DAGA 2025*. Version v25.04 <https://doi.org/10.5281/zenodo.15112633>. Mar. 2025.
- [2] E. Sarradj, C. Fritzsche, and T. Geyer. “Silent Owl Flight: Bird Flyover Noise Measurements”. In: *AIAA Journal* 49.4 (2011), pp. 769–779.
- [3] H. Brick, T. Kohrs, E. Sarradj, and T. Geyer. “Noise from high-speed trains: Experimental determination of the noise radiation of the pantograph”. In: *Proceedings of Forum Acusticum*. Aalborg, Denmark: Spanish Acoustical Society, 2011.
- [4] E. Sarradj and G. Herold. “A Python framework for microphone array data processing”. In: *Applied Acoustics* 116 (2017), pp. 50–58.
- [5] F. Gstrein, B. Zang, Y. Mayer, and M. Azarpeyvand. “Airfoil Trailing-Edge Noise Reduction by Application of Finlets”. In: *AIAA Journal* 60.1 (2022), pp. 236–248.
- [6] A. Celik, Y. Mayer, and M. Azarpeyvand. “On the aeroacoustic characterization of a robust trailing-edge serration”. In: *Physics of Fluids* 33.7, 075120 (2021).
- [7] Y. Mayer, B. Zang, and M. Azarpeyvand. “Aeroacoustic investigation of an oscillating airfoil in the pre- and post-stall regime”. In: *Aerospace Science and Technology* 103, 105880 (2020).
- [8] S. Palreja-Cabre, P. Chaitanya, P. Joseph, J. Kim, M. Priddin, L. Ayton, T. Geyer, and T. Chong. “Downstream porosity for the reduction of turbulence-aerofoil interaction noise”. In: *Journal of Sound and Vibration* 541, 117324 (2022).
- [9] T. Zhang, T. Geyer, C. De Silva, J. Fischer, C. Doolan, and D. Moreau. “Experimental Investigation of Tip Vortex Formation Noise Produced by Wall-Mounted Finite Airfoils”. In: *Journal of Aerospace Engineering* 34.6, 04021079 (2021).
- [10] T. Zhang, D. Moreau, T. Geyer, J. Fischer, and C. Doolan. “Dataset on tip vortex formation noise produced by wall-mounted finite airfoils with sinusoidal and porous tip geometries”. In: *Data in Brief* 30, 105471 (2020).
- [11] R. Zamponi, P. Chiariotti, G. Battista, C. Schram, and P. Castellini. “3D Generalized Inverse Beamforming in wind tunnel aeroacoustic testing: application to a Counter Rotating Open Rotor aircraft model”. In: *Applied Acoustics* 163, 107229 (2020).
- [12] Y. Schulz, A. Mattar, T. Hehn, and J. Kooij. “Hearing What You Cannot See: Acoustic Vehicle Detection around Corners”. In: *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 2587–2594.
- [13] L. Feng, M. Zan, L. Huang, and Z. Xu. “A double-step grid-free method for sound source identification using deep learning”. In: *Applied Acoustics* 201, 109099 (2022).
- [14] G. Zhou, H. Liang, X. Ding, Y. Huang, X. Tu, and S. Abbas. “Acoustic-Net: A Novel Neural Network for Sound Localization and Quantification”. In: *arXiv preprint arXiv:2203.16988* (2022).
- [15] C. Zhang and W. Ma. “Efficient computation of modal transfer function in mode composition beamforming for rotating broadband source localization”. In: *Journal of the Acoustical Society of America* 152.6 (2022), pp. 3422–3428.
- [16] Y. Wang, C. Zhang, X. Liu, and W. Ma. “Analysis of error induced by the Doppler effect in the ROSI beamforming for rotating sound source identification”. In: *Aerospace Systems* 5 (2022), pp. 323–330.
- [17] C. Lee, C. Zhang, and W. Ma. “Effects of radial misalignment on virtual rotating array beamforming for rotating sound source localization”. In: *Aerospace Systems* (2022).
- [18] W. D. A. Fonseca, P. H. Mareze, F. R. de Mello, and C. C. da Fonseca. “Teaching acoustical beamforming via active learning”. In: *Proceedings on CD of the 9th Berlin Beamforming Conference*. Berlin, Germany: Gesellschaft zur Förderung angewandter Informatik (GFAI), 2022, pp. 1–21.
- [19] E. Sarradj, G. Herold, P. Sijtsma, R. Merino-Martinez, T. F. Geyer, C. J. Bahr, R. Porteous, D. Moreau, and C. J. Doolan. “A microphone array method benchmarking exercise using synthesized input data”. In: *23rd AIAA/CEAS Aeroacoustics Conference*. Denver, Colorado, June 2017, AIAA 2017-3719.
- [20] O. Lylloff, G. Herold, A. Kujawski, and E. Sarradj. “State of open-source software for microphone array processing”. In: June 2024.
- [21] O. Lylloff. “AeroAcoustics.jl: A Julia package for aeroacoustics”. In: *Journal of Open Source Software* 9.97 (2024), p. 6390.